Recursive Function to Generate the First n Fibonacci Numbers

with Counting of Recursive Calls

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

1. Problem Statement

Write a recursive function to generate the first n Fibonacci numbers. Also count the number of recursive calls made.

2. Objective

- To understand and implement recursion using the Fibonacci sequence.
- To identify the base cases and the recursive case of a recursive function.
- To count the number of function calls made during execution using a global counter.
- To analyse why simple recursive Fibonacci becomes slow as n grows.

3. Theory

3.1 Fibonacci Sequence

The Fibonacci sequence is a series in which every number is the sum of the two numbers before it. The sequence starts with 0 and 1:

$$\mathbf{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots}$$

Mathematically, it is defined as:

$$\mathbf{F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad for\ n \geq 2}$$

3.2 Recursion

Recursion is a technique in which a function calls itself to solve a smaller version of the same problem. Every recursive function needs:

- **Base case(s)** – the condition where the function stops calling itself. Here: n = 0 returns 0 and n = 1 returns 1.
- **Recursive case** – the part where the function calls itself with a smaller input. Here: fib(n − 1) + fib(n − 2).

3.3 Counting the Recursive Calls

A global variable call_count is declared and set to 0. The very first statement inside fib() is call_count++, so the counter increases by one every time the function is entered. Before computing each term, the counter is reset to 0, so after the call returns it holds the number of calls needed for that term. These values are added up to get the total.

4. Algorithm

Step 1: Start the program and declare a global variable call_count = 0.

Step 2: Read the number of terms n from the user. If $n \leq 0$, display an error message and stop.

Step 3: Set total_calls = 0.

Step 4: Repeat for i = 0 to $n - 1$: (a) set call_count = 0; (b) compute value = fib(i); (c) add call_count to total_calls; (d) print i, value and call_count.

Step 5: Inside fib(n): increase call_count by 1; if n = 0 return 0; if n = 1 return 1; otherwise return fib(n − 1) + fib(n − 2).

Step 6: Print total_calls and stop.

5. Source Code (C Language)

```
#include <stdio.h>

/* Global counter: incremented on every call to fib() */
long long call_count = 0;

/* Recursive function to return the nth Fibonacci number
   F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)            */
long long fib(int n)
{
    call_count++;                    /* count this call */

    if (n == 0)                      /* base case 1 */
        return 0;
    if (n == 1)                      /* base case 2 */
        return 1;

    return fib(n - 1) + fib(n - 2);  /* recursive case */
}

int main(void)
{
    int n, i;
    long long total_calls = 0;

    printf("Enter the value of n: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Please enter a positive integer.\n");
        return 1;
    }

    printf("\nFirst %d Fibonacci numbers:\n", n);
    printf("%-8s %-16s %-16s\n", "Index", "Fibonacci", "Calls made");
    printf("-----------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        call_count = 0;                        /* reset for each term */
        long long value = fib(i);              /* recursive call      */
        total_calls += call_count;             /* accumulate          */
        printf("%-8d %-16lld %-16lld\n", i, value, call_count);
    }

    printf("-----------------------------------------\n");
    printf("Total number of recursive calls = %lld\n", total_calls);
```

```
    return 0;
}
```

Program: fibonacci.c

6. Sample Output

6.1 Test Case 1: n = 5

```
Enter the value of n: 5

First 5 Fibonacci numbers:
Index    Fibonacci        Calls made
-------------------------------------------
0        0                1
1        1                1
2        1                3
3        2                5
4        3                9
-------------------------------------------
Total number of recursive calls = 19
```

Output for n = 5

6.2 Test Case 2: n = 10

```
Enter the value of n: 10

First 10 Fibonacci numbers:
Index    Fibonacci        Calls made
-------------------------------------------
0        0                1
1        1                1
2        1                3
3        2                5
4        3                9
5        5                15
6        8                25
7        13               41
8        21               67
9        34               109
-------------------------------------------
Total number of recursive calls = 276
```

Output for n = 10

6.3 Invalid Input (n = 0)

```
Enter the value of n: 0
Please enter a positive integer.
```

Output for invalid input

7. Explanation and Analysis of Recursive Calls

7.1 Working of the Program

The loop in main() asks for Fibonacci numbers at index 0, 1, 2, ... n − 1. For each index the function fib() is called, and it keeps calling itself until the base cases are reached. The values then return back up and are added to give the final result.

7.2 Recursion Tree for fib(4)

The tree below shows every call made while computing fib(4). Counting each line gives 9 calls, which matches the output of the program (Calls made = 9 at index 4).

```
fib(4)
├── fib(3)
│   ├── fib(2)
│   │   ├── fib(1)   -> 1
│   │   └── fib(0)   -> 0
│   └── fib(1)       -> 1
└── fib(2)
    ├── fib(1)       -> 1
    └── fib(0)       -> 0
```

Recursion tree of fib(4): 9 calls in total

Notice that fib(2) is calculated twice and fib(1) three times. The same sub-problems are solved again and again, which is the main reason this method is slow for large n.

7.3 Number of Calls for Each Term

Let C(n) be the number of calls made by fib(n). The call to fib(n) itself counts as 1, plus the calls made by fib(n − 1) and fib(n − 2):

$$\mathbf{C(0) = C(1) = 1, \quad C(n) = C(n-1) + C(n-2) + 1}$$

Solving this gives the closed form $C(n) = 2 \cdot F(n+1) - 1$. The table below verifies it for n = 0 to 9 and agrees with the program output.

n	F(n)	Calls for fib(n)	Check: 2·F(n+1) − 1
0	0	1	1
1	1	1	1
2	1	3	3
3	2	5	5
4	3	9	9
5	5	15	15
6	8	25	25
7	13	41	41
8	21	67	67
9	34	109	109
Total (n = 10 terms)		**276**	

Table 1: Number of recursive calls for each term

7.4 Time and Space Complexity

- **Time complexity:** $O(2^n)$ (more precisely $O(\varphi^n)$, where $\varphi \approx 1.618$). The number of calls grows almost like the Fibonacci numbers themselves.
- **Space complexity:** O(n), because the maximum depth of the recursion (call stack) is n.

8. Conclusion

The program successfully generates the first n Fibonacci numbers using a recursive function and counts the number of recursive calls using a global counter. For n = 10, the program prints the series 0 to 34 and makes a total of 276 recursive calls, while for n = 5 it makes 19 calls. The results show that the number of calls grows very fast, because the same values are recomputed many times. This makes plain recursion suitable for learning and small values of n, whereas for large n an improved approach such as memoization or an iterative method should be used.